

MACHINE LEARNING IN FINANCE

MODULE 1 | LESSON 3

SUPERVISED MODELS: CLASSIFICATION

Reading Time	60 minutes
Prior Knowledge	Linear regression, Hyperparameters
Keywords	logistic regression, confusion matrix

In the last two lessons, we explored the supervised linear regression model. In this lesson, we will turn our attention to classification methods.

1. Introduction to Classification Models

In Lesson 1, we mentioned that the most common supervised learning tasks are regression (predicting values) and classification (predicting classes). In the first two lessons, we explored a regression task, predicting stock returns, using linear regression, and using regularization methods. Now, we will turn our attention to classification methods.

1.1 Time Series Momentum: Thresholds

Let's consider the return prediction problem we studied in Lesson 1. Instead of trying to predict the realization of returns, we are now only interested in predicting whether the market is going up or down. Thus, we are going to classify the market into categories of positive return and negative return.

Next, as we did in previous lessons, we upload the necessary information to train and evaluate the prediction model.

```
In [ ]: import numpy as np
import yfinance as yf

# Getting historical market data from SPY (ETF) (SPY)
df = yf.download("SPY", start="2000-01-01", end="2022-01-01")

df["Ret"] = df["Adj Close"].pct_change()

name = "Ret"
df["Ret10_i"] = (
    df[name].rolling(10).apply(lambda x: 100 * ((np.prod(1 + x)) ** (1 / 10) - 1
)
```

```

df["Ret25_i"] = (
    df[name].rolling(25).apply(lambda x: 100 * ((np.prod(1 + x)) ** (1 / 25) - 1)
)
df["Ret60_i"] = (
    df[name].rolling(60).apply(lambda x: 100 * ((np.prod(1 + x)) ** (1 / 60) - 1)
)
df["Ret120_i"] = (
    df[name].rolling(120).apply(lambda x: 100 * ((np.prod(1 + x)) ** (1 / 120) - 1)
)
df["Ret240_i"] = (
    df[name].rolling(240).apply(lambda x: 100 * ((np.prod(1 + x)) ** (1 / 240) - 1)
)

del df["Open"]
del df["Close"]
del df["High"]
del df["Low"]
del df["Volume"]
del df["Adj Close"]

df = df.dropna()
df.tail(10)

```

```

In [ ]: df["Ret25"] = df["Ret25_i"].shift(-25)
df = df.dropna()
df.tail(10)

```

We now transform our previous continuous label y_t into a $\{-1, 1\}$ variable that indicates whether the market has gone, respectively, down or up over a 25-day interval.

```

In [ ]: df["Output"] = df["Ret25"].apply(np.sign)
del df["Ret25"]
df.tail(10)

```

```

In [ ]: df.describe()

```

Split the information into a training set and test set. Below, we display the times series of the label we want to predict.

```

In [ ]: X, y = df.iloc[:, 0:-1], df.iloc[:, -1]
print(X.shape, y.shape)
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=int(len(y) * 0.5), shuffle=False
)
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)

```

```

In [ ]: import matplotlib.pyplot as plt

plt.figure(figsize=(15, 7))
plt.plot(
    df.index,
    np.cumsum(y),
    label="Cumulative ups (+1) and downs (-1) of SPY over 25 days",
)
legend = plt.legend(loc="upper left")

```

```
# plt.ylim([-1.25,1.25])
plt.show()
```

1.2 Scaling

One important step before training an ML model involves data pre-processing. We will cover several techniques in the last module, but here, we are going to introduce one of the most important steps: *feature scaling*. Machine Learning algorithms perform badly when the input features have very different scales, which tends to slow down and worsen the performance of the optimization algorithms. There are two common ways to get all attributes to have the same scale: min-max scaling and standardization.

Min-max scaling (many people call this normalization) is quite simple: Values are shifted and rescaled so that they end up ranging from 0 to 1. We do this by subtracting the min value and dividing by the max minus the min. Scikit-Learn provides a transformer called `MinMaxScaler` for this. Formally, each feature x_j of instance i , $x_j^{(i)}$, is transformed as:
$$x_j^{(i)} \mapsto \frac{x_j^{(i)} - \min_j}{\max_j - \min_j}$$
 where $\min_j$ and $\max_j$ denote, respectively, the minimum and maximum values of the j th feature.

Standardization is quite different: First, it subtracts the mean value (so standardized values always have a zero mean), and then it divides by the variance so that the resulting distribution has a unit variance. Unlike min-max scaling, standardization does not bound values to a specific range, which may be a problem for some algorithms. Scikit-Learn provides a transformer called `StandardScaler` for standardization.

Another form of feature pre-processing is referred to as *whitening*, in which the axis-system is rotated to create a new set of de-correlated features, each of which is scaled to unit variance. Typically, Principal Component Analysis (a method of Unsupervised ML) is used to achieve this goal.

It is important to fit the scalers to the training data only, not to the full dataset (including the test set). Only then can you use them to transform the training set and the test set (and new data).

The block of code below scales the observations in our data.

```
In [ ]: from sklearn.preprocessing import MinMaxScaler

scaler_input = MinMaxScaler(feature_range=(-1, 1))
scaler_input.fit(X_train)
X_train = scaler_input.transform(X_train)
X_test = scaler_input.transform(X_test)
```

2. Logistic Regression Model

Logistic Regression (also called Logit Regression) is commonly used to estimate the probability that an instance belongs to a particular class (e.g., what is the probability that the market will go up?). If the estimated probability is greater than 50%, then the model

predicts that the instance belongs to that class (called the positive class, labeled "1"), or else it predicts that it does not (i.e., it belongs to the negative class, labeled "0"). This makes it a binary classifier.

Suppose that we have a training sample with instances where the labels y simply denote if the market has gone up $y=1$ or gone down $y=0$. Just like a linear regression model, a logistic regression model computes a weighted sum of the input features (plus a bias term), but instead of outputting the result directly like the Linear Regression model does, it outputs the logistic of this result:
$$\widehat{p}^{(i)} = \sigma(x^{(i)\top} \theta)$$
 where $\sigma(\cdot)$ is the sigmoid function defined as:
$$\sigma(z) = \frac{1}{1 + \exp(-z)}$$
 Then, the model's prediction from the Logistic Regression is given by:
$$\widehat{y}^{(i)} = \begin{cases} 0 & \text{if } \widehat{p}^{(i)} < 0.5 \\ 1 & \text{if } \widehat{p}^{(i)} \geq 0.5 \end{cases}$$
 Notice that the Logistic Regression then predicts 1 if $x^{(i)\top} \theta$ is positive and 0 otherwise.

2.1 Training and Cost Function

The objective of training is to set the parameter vector θ so that the model estimates high probabilities for positive instances ($y^{(i)} = 1$) and low probabilities for negative instances ($y^{(i)} = 0$). This idea is captured by the following cost function:
$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(\widehat{p}^{(i)}) + (1 - y^{(i)}) \log(1 - \widehat{p}^{(i)}) \right]$$
 This cost function makes sense because $-\log(\widehat{p}^{(i)})$ grows very large when $\widehat{p}^{(i)}$ approaches 0, so the cost will be large if the model estimates a probability close to 0 for a $y^{(i)} = 1$, and similarly for $y^{(i)} = 0$ if the negative when $\widehat{p}^{(i)}$ approaches 1. Indeed, in Econometrics, the previous function is nothing more than the log-likelihood function for a Logit model.

While there is no closed form for the vector θ that maximizes the cost function above, we know that the function is convex and, for instance, Gradient Descent is guaranteed to find the global minimum as long as the learning rate is small enough and we wait for enough time. Thus, if we opt to use the Gradient Descent algorithm (either Batch, Mini-batch, or Stochastic) we would use the following partial derivatives from the cost function:
$$\frac{\partial}{\partial \theta_j} J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} - \sigma(x^{(i)\top} \theta) \right] x_j^{(i)}$$
 Just like in the linear regression, logistic regression can be regularized using the ridge and lasso penalties we described in Lesson 2. Scikit-Learn actually adds an ℓ_2 (Ridge) penalty by default.

Let's train our stock market prediction model using time series information with a Logistic Regression.

Note: If we were interested in playing with the regularization strength, we should include as parameter the input "C" in the class `LogisticRegr` defined below, where "C" is an inverse of the regularization strength.

```
In [ ]: from sklearn.linear_model import LogisticRegression

# all parameters not specified are set to their defaults
logisticRegr = LogisticRegression()
logisticRegr.fit(X_train, y_train)
```

```
In [ ]: predictions = logisticRegr.predict(X_test)
```

```
In [ ]: # Use score method to get accuracy of model
score = logisticRegr.score(X_test, y_test)
print(score)
```

The model is able to predict the actual outcome 72% of the time. It seems a quite considerable figure given the simplicity of the model, right? Below, we show that this figure can be quite misleading.

Before describing in detail the performance measures of classifiers, we first describe a simple generalization of logistic regression to a multiclass setup.

2.2 Softmax Regression

The logistic regression model generalizes to multiple classes, without having to train and combine multiple binary classifiers. This is called *Softmax Regression* or *Multinomial Logit Regression*. For instance, imagine that we want to add a new category to our return prediction model that captures small movements in the returns of the market, say in the $[-.5\%, +.5\%]$ range. Thus, we can train a model that predicts the probability that the market return is "high" ($>.5\%$), low ($<-.5\%$), or "intermediate".

The Softmax Regression model first computes a score for each instance i and category c , $x^{(i)} \theta^{(c)}$, and then estimates the probability of each class by using the softmax function. Notice that each category has a specific vector of parameters, $\theta^{(c)}$, with dimension equal to the number of input features, plus a bias term.

Then, if we have C different categories, the estimated probability of category c for instance i is given the exponential of its score, normalized by the sum of the exponential of the scores across categories.
$$\widehat{p}_{c^{(i)}} = \text{Softmax}_c \left(x^{(i)}, \Theta \right) = \frac{\exp \left(x^{(i)} \theta^{(c)} \right)}{\sum_{j=1}^C \exp \left(x^{(i)} \theta^{(j)} \right)}$$
 where Θ collects all the parameters of the model. The model prediction $\widehat{y}^{(i)}$ will be the category with the highest score $x^{(i)} \theta^{(c)}$.

In analogy with the logistic regression, the cost function to train the parameters of the model can be defined as:
$$J(\Theta) = - \frac{1}{n} \sum_{i=1}^n \sum_{c=1}^C y^{(i)} \log \left(\widehat{p}_{c^{(i)}} \right)$$

$$-\frac{1}{m} \sum_{c=1}^C \sum_{i=1}^m y^{(i)} \log(\widehat{p}_c^{(i)})$$
 which is generally called the *cross-entropy* cost function. The corresponding elements of the gradient vector are:

$$\frac{\partial}{\partial \theta_k^{(c)}} J(\Theta) = -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} - \text{Softmax}_c(x^{(i)}, \Theta) \right] x_k^{(i)}$$
 From which we can use Gradient Descent optimization.

3. Performance Measures in Classification

Evaluating a classifier is often significantly trickier than evaluating a regression model. One simple measure of performance may simply be the frequency with which the classifier is successful at classifying the labels in the test sample, i.e., its *accuracy*. However, this indicator may be misleading. In our stock market prediction model, we found a 72% success rate in predicting whether the stock market is going up or down. However, such predictive power may arise simply from the model predicting most of the time that the stock market is going up, which happens with roughly 72% probability. This is the actual case, as we are going to show below.

Similarly, our classification model may target the prediction of relatively rare classes or events, say a 5% probability event or class. In those instances, even a terrible classifier may achieve high accuracy. For instance, if we always predict that the event is never happening we are going to reach by default a 95% accuracy. To overcome this and improve the selection of classifier models, there are many performance measures available that we outline below.

3.1 Confusion Matrix

A much better way to evaluate the performance of a classifier is to look at the confusion matrix. The general idea is to count the number of times instances of category A are classified as category B. To compute the confusion matrix, you first need to have a set of predictions so they can be compared to the actual targets. Each row in a confusion matrix represents an actual class, while each column represents a predicted class. A perfect classifier would have only true positives and true negatives, so its confusion matrix would have nonzero values only on its main diagonal (top left to bottom right).

In the case of a two-category classifier where 0s are the negative class and 1s are the positive class, we can represent the confusion matrix as:

$$\begin{matrix} & \text{Actual Negatives} & \text{Actual Positives} \\ \text{Predicted Negatives} & \text{True Negatives} & \text{False Positives} \\ \text{Predicted Positives} & \text{False Negatives} & \text{True Positives} \end{matrix}$$
 Let's observe the confusion matrix we obtain from our classifier.

```

In [ ]: # import matplotlib.pyplot as plt
import seaborn as sns
from sklearn import metrics

cm = metrics.confusion_matrix(y_test, predictions)
plt.figure(figsize=(9, 9))
sns.heatmap(cm, annot=True, fmt=".3f", linewidths=0.5, square=True, cmap="Blues_")
plt.ylabel("Actual label")
plt.xlabel("Predicted label")
  
```

```
all_sample_title = "Accuracy Score: {0}".format(score)
plt.title(all_sample_title, size=15);
```

Indeed, the 72% success rate of the model comes from it predicting that the stock market is going up, which takes place with a probability that is close to that success rate. Thus, we can assess that our model is a poor classifier to predict the returns of the stock market.

3.2 Precision and Recall

The extent of the positive predictions can be obtained through the precision of the classifier:
$$\text{Precision} = \frac{TP}{TP + FP}$$
 where TP is the total number of true positives and FP is the number of false positives. That is, the denominator contains the number of times that the classifier has predicted a 1, so precision captures the proportion of accurate predictions over all the 1s that have been predicted by the model.

Precision is usually combined with another metric, recall:
$$\text{Recall} = \frac{TP}{TP + FN}$$
 Recall captures the proportion of accurate predictions over all the 1s that appear in the sample.

Precision and recall are usually combined in a single metric that is named F_1 score, a harmonic mean of both values that gives much more weight to low values. Thus, the F_1 score will only be high if both precision and recall are high:
$$F_1 = 2 \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

The F_1 score favors classifiers with similar precision and recall. Sometimes, we do not want that. For instance, if we develop a model to detect the probability of a large drop (crash) in the stock market, in which case the category is 1, we may be willing to sacrifice some accuracy, predict the crash when it does not take place, against some recall, do not predict the crash when the crash actually happens.

Unfortunately, we cannot have classifiers with both high precision and high recall at the same time. To see this, notice that models like logistic regression return probabilities rather than discrete outputs. To determine which class the model is predicting, we set a threshold value on the predicted probabilities to distinguish between a positive and a negative class, such as 0 or 5 in the Logistic Regression above. Depending on the threshold value, the predicted class of some observations may change. If a classifier sets a high threshold to increase precision, it is going to increase the number of positive cases that are going to be predicted as negative, reducing recall. Otherwise, if a classifier sets a high bar to increase recall, it is going to be at the cost of predicting more negative cases as positives, reducing precision.

3.3 ROC and AUC

The Receiver Operating Characteristic (ROC) curve is another common tool used with binary classifiers. The ROC plots the recall of a classifier against the *false positive rate*, the proportion of 0s that are classified as 1s. As in the precision-recall trade-off, when a

classifier increases its recall, then it is going to generate more false positives. One way to compare classifiers is to use the Area Under the Curve (AUC), which gives us the area that lies below the ROC of a classifier. For a perfect classifier, the AUC is one. For a purely random classifier, the AUC is 0.5.

What metric should we use to determine what is a good classifier? We should use the F_1 when we care more about the false positives or when the positive class is rare.

The block of code below plots the ROC curve for our logistic regression model. To obtain this, the `roc_curve()` function computes the recall, or True Positive Rate, and the False Positive Rate in the test sample for various threshold values that determine the prediction of the model. Notice how the predictive ability of the model is *among the poorest of the poor*.

```
In [ ]: from sklearn import metrics

fpr, tpr, thresholds = metrics.roc_curve(y_test, predictions)
roc_auc = metrics.auc(fpr, tpr)
display = metrics.RocCurveDisplay(
    fpr=fpr, tpr=tpr, roc_auc=roc_auc, estimator_name="estimation"
)
display.plot()
plt.show()
```

4. Conclusion

With this, we finish the description of basic classification problems in Machine Learning in the context of logistic regression. In Lesson 4, we will develop an application for credit scoring.

See you there!